RONIN`S PATH

Código explicado • 4º bimestre

Resumo dos requisitos e leitura linha por linha das funções de combate.

01 Ataque corpo a corpo

Z inicia a katana. A área da esquerda ou da direita fica ativa apenas durante a janela de dano. Um conjunto de identificadores impede acertar o mesmo alvo várias vezes no mesmo golpe.

02 Pelo menos três tipos de projéteis

1, 2 e 3 selecionam o tipo; X dispara. Cada índice escolhe velocidade e recarga. O terceiro recebe impulso vertical e causa explosão em área. O dano é um campo configurado no componente do projétil.

03 Inimigo dispara por proximidade

O guarda compara a distância com seu alcance e testa obstáculos na linha de visão. Antes de atirar, mostra uma preparação. O chefe combina ataque distante e golpe próximo em uma máquina de estados.

04 Defesa com tecla e animação

Segurar F solicita defesa. O estado ativa o parâmetro defendendo do Animator. O dano identificado como projétil é bloqueado; o golpe próximo do chefe não é bloqueado por essa regra.

05 Objetos destrutíveis reagem ao combate

AlvoTemplo centraliza vida e dano. Ao chegar a zero, desliga colisores, libera um prêmio quando configurado, avisa os ouvintes do evento de derrota e destrói o objeto.

Edição de estudo • 04/11/2026 • Unity 2017.4.40f1
35 funções/propriedades selecionadas em 7 scripts. Código após organização da indentação.

Como ler este guia

Cada quadro apresenta o número da linha no arquivo, o código original e uma explicação em português. Os recuos mostram quais comandos pertencem a cada condição ou repetição. Linhas longas são quebradas apenas para caber na página; o número continua sendo o da linha original.

As chaves abrem e fecham blocos. Linhas em branco são omitidas das tabelas. Quando um if ou foreach não tem chaves, somente a instrução seguinte pertence a ele. O recuo ajuda a leitura, mas são as chaves e a sintaxe que definem a execução.

Controles relacionados

Z: ataque de katana. F mantido: defesa. X mantido: tentativas de disparo respeitando a recarga.

1 / 2 / 3: seleção dos projéteis. Q: dash, citado porque também impede atacar e defender ao mesmo tempo.

Vocabulário essencial

bool / int / float: verdadeiro ou falso; número inteiro; número decimal. O sufixo f indica float. void não devolve valor; return encerra a função e pode devolver um resultado.

= atribui; == compara; != significa diferente. && exige ambas as condições; || aceita uma delas; ! nega. Em condição ? A : B, escolhe A se verdadeiro, ou B caso contrário.

null: ausência de referência. this: a instância atual. GetComponent busca um componente. Instantiate cria uma cópia; Destroy solicita sua remoção.

Time.time: relógio do jogo em segundos. Time.deltaTime: duração do quadro. Vector2/Vector3: coordenadas ou direções. Distâncias usam unidades do mundo do Unity.

Trigger: área que detecta sobreposição. Rigidbody2D: corpo da física 2D. Animator: controlador de estados de animação. HashSet: conjunto sem elementos duplicados.

Valores e escopo

Campos públicos ou serializados podem receber valores no Inspector diferentes dos padrões escritos no script. Números fixos explicados aqui são os presentes nas funções. Este guia detalha o fluxo das cinco exigências; rotinas auxiliares externas aos trechos são identificadas pelo seu papel, sem reproduzir todos os scripts do jogo.

1. Ataque corpo a corpo

Z inicia a katana. A área da esquerda ou da direita fica ativa apenas durante a janela de dano. Um conjunto de identificadores impede acertar o mesmo alvo várias vezes no mesmo golpe.

Samurai.cs • AtualizarCombate

Lê as teclas, atualiza a duração do ataque e habilita a área correta da katana.

Nº	CÓDIGO	O QUE A LINHA FAZ
528	`void AtualizarCombate()`	Declara a função AtualizarCombate. Lê as teclas, atualiza a duração do ataque e habilita a área correta da katana.
529	`{`	Abre este bloco de instruções.
530	`ataqueAtivo = Time.time < fimAtaque && !missaoEncerrada;`	Mantém o ataque ativo enquanto o relógio não chegou ao fim e a missão continua.
531	`VerificarBonusAndar();`	Atualiza o bônus associado a andar; é uma rotina de apoio ao combate.
532	`if (feedbackDano && Renderer != null)`	Só muda a cor de dano se esse efeito estiver ativo e houver um SpriteRenderer.
533	`{`	Abre este bloco de instruções.
534	`Renderer.color = Time.time < invulneravelAte && ((int)(Time.time * 12) % 2 == 0) ? new Color(1, .4f, .4f, .6f) : corAntesDano;`	Alterna entre vermelho semitransparente e a cor salva durante a invulnerabilidade. O resto da divisão por 2 cria o piscar.
535	`if (Time.time >= invulneravelAte)`	Verifica se terminou o período de invulnerabilidade.
536	`feedbackDano = false;`	Desliga o controle do efeito visual de dano.
537	`}`	Fecha o bloco deste nível de recuo.
538	`DefinirDefesa(Input.GetKey(KeyCode.F));`	Envia verdadeiro para a defesa enquanto F estiver pressionado; ao soltar, envia falso.
539	`if (Input.GetKeyDown(KeyCode.Q))`	Detecta apenas o instante em que Q é pressionado.

Nº	CÓDIGO	O QUE A LINHA FAZ
540	`            IniciarDash();`	Solicita o dash. A própria rotina verifica se pode executá-lo.
541	`        if (Input.GetKeyDown(KeyCode.Z))`	Detecta o início do pressionamento de Z.
542	`            AtacarKatana();`	Solicita o ataque de katana.
543	`        if (Input.GetKeyDown(KeyCode.Alpha1))`	Detecta o pressionamento da tecla 1.
544	`            tipoProjetil = 0;`	Seleciona o primeiro projétil, cujo índice interno é zero.
545	`        if (Input.GetKeyDown(KeyCode.Alpha2))`	Detecta o pressionamento da tecla 2.
546	`            tipoProjetil = 1;`	Seleciona o segundo projétil, índice 1.
547	`        if (Input.GetKeyDown(KeyCode.Alpha3))`	Detecta o pressionamento da tecla 3.
548	`            tipoProjetil = 2;`	Seleciona o terceiro projétil, índice 2.
549	`        if (Input.GetKey(KeyCode.X))`	Testa continuamente se X está pressionado.
550	`            DispararMissao();`	Tenta disparar. O intervalo entre tiros é controlado dentro dessa função.
551	`        if (areaKatanaDireita != null)`	Confere se a área de ataque à direita foi configurada.
552	`            areaKatanaDireita.enabled = JanelaKatana && !Renderer.flipX;`	Ativa essa área somente na janela de dano e quando o personagem olha à direita.
553	`        if (areaKatanaEsquerda != null)`	Confere se a área de ataque à esquerda foi configurada.
554	`            areaKatanaEsquerda.enabled = JanelaKatana && Renderer.flipX;`	Ativa essa área somente na janela de dano e quando a imagem está virada à esquerda.
555	`        tipoProjetil = Mathf.Clamp(tipoProjetil, 0, 2);`	Limita a seleção aos índices válidos 0, 1 e 2.
556	`    }`	Fecha o bloco deste nível de recuo.

Samurai.cs • JanelaKatana

Propriedade de leitura que informa se a espada pode causar dano naquele instante.

Nº	CÓDIGO	O QUE A LINHA FAZ
452	`public bool JanelaKatana`	Declara uma propriedade bool pública: ao ser consultada, fornece verdadeiro ou falso.
453	`{`	Abre este bloco de instruções.
454	`    get`	Inicia o acessor de leitura desta propriedade; a consulta executa o bloco a seguir.
455	`    {`	Abre este bloco de instruções.
456	`        return ataqueAtivo && Time.time >= inicioAtaque + 1f / 3f && Time.time < fimAtaque - .02f;`	Retorna verdadeiro entre aproximadamente 0,333 e 0,48 segundo após o início, enquanto o ataque está ativo. A espada não causa dano durante toda a animação.
457	`    }`	Fecha o bloco deste nível de recuo.
458	`}`	Fecha o bloco deste nível de recuo.

Samurai.cs • AtacarKatana

Valida o pedido de ataque e inicia seus temporizadores. Retorna bool: começou ou não começou.

Nº	CÓDIGO	O QUE A LINHA FAZ
619	`public bool AtacarKatana()`	Declara a função AtacarKatana. Valida o pedido de ataque e inicia seus temporizadores. Retorna bool: começou ou não começou.
620	`{`	Abre este bloco de instruções.
621	`    if (!(combateHabilitado || modoMissao) || missaoEncerrada || defesaAtiva || dashAtivo || carregando != null || Time.time < proximoAtaque)`	Recusa a katana se não houver combate, a missão acabou, há defesa ou dash, uma caixa está sendo carregada ou o intervalo ainda não terminou.
622	`        return false;`	Retorna falso para informar que nenhum ataque começou.

Nº	CÓDIGO	O QUE A LINHA FAZ
623	`inicioAtaque = Time.time;`	Salva o instante de início do ataque.
624	`fimAtaque = Time.time + .5f;`	Agenda seu fim para meio segundo depois.
625	`proximoAtaque = Time.time + .58f;`	Permite o próximo ataque somente após 0,58 segundo.
626	`ataqueAtivo = true;`	Marca o personagem como atacando.
627	`alvosKatana.Clear();`	Esvazia o conjunto de alvos atingidos no ataque anterior.
628	`SomRonin.Tocar("katana", transform.position);`	Reproduz o efeito de katana na posição do jogador.
629	`return true;`	Retorna verdadeiro para confirmar o início do ataque.
630	`}`	Fecha o bloco deste nível de recuo.

Samurai.cs • AcertarKatana

Recebe um AlvoTemplo e impede golpes repetidos no mesmo alvo durante uma única animação.

Nº	CÓDIGO	O QUE A LINHA FAZ
632	`public void AcertarKatana(AlvoTemplo alvo)`	Declara a função AcertarKatana. Recebe um AlvoTemplo e impede golpes repetidos no mesmo alvo durante uma única animação.
633	`{`	Abre este bloco de instruções.
634	`if (JanelaKatana && alvo != null && alvosKatana.Add(alvo.GetInstanceID()))`	Exige a janela de dano, um alvo válido e um identificador ainda não registrado. Add retorna falso quando esse alvo já foi atingido.
635	`alvo.ReceberDano(1, this);`	Aplica 1 ponto de dano, passando este Samurai como autor.
636	`}`	Fecha o bloco deste nível de recuo.

AreaKatana.cs • OnTriggerEnter2D

Evento do Unity chamado quando um colisor entra no trigger da espada.

Nº	CÓDIGO	O QUE A LINHA FAZ
6	`void OnTriggerEnter2D(Collider2D c)`	Declara a função OnTriggerEnter2D. Evento do Unity chamado quando um colisor entra no trigger da espada.
7	`{`	Abre este bloco de instruções.
8	`    Tocar(c);`	Encaminha o colisor que acabou de entrar na área para a rotina comum de contato.
9	`}`	Fecha o bloco deste nível de recuo.

AreaKatana.cs • OnTriggerStay2D

Evento do Unity usado enquanto um colisor permanece sobreposto ao trigger da espada.

Nº	CÓDIGO	O QUE A LINHA FAZ
11	`void OnTriggerStay2D(Collider2D c)`	Declara a função OnTriggerStay2D. Evento do Unity usado enquanto um colisor permanece sobreposto ao trigger da espada.
12	`{`	Abre este bloco de instruções.
13	`    Tocar(c);`	Reavalia o colisor que permanece dentro da área, permitindo acertar quando a janela de dano abrir.
14	`}`	Fecha o bloco deste nível de recuo.

AreaKatana.cs • Tocar

Conecta o contato físico da espada à lógica de dano do jogador.

Nº	CÓDIGO	O QUE A LINHA FAZ
16	`void Tocar(Collider2D c)`	Declara a função Tocar. Conecta o contato físico da espada à lógica de dano do jogador.
17	`{`	Abre este bloco de instruções.
18	`if (jogador != null)`	Confere se esta área conhece o Samurai responsável pelo ataque.
19	`jogador.AcertarKatana(AlvoTemplo.Identificar(c));`	Converte o colisor em alvo de combate e pede ao Samurai para tentar acertá-lo. Um alvo inválido será ignorado.
20	`}`	Fecha o bloco deste nível de recuo.

AlvoTemplo.cs • Identificar

Transforma um Collider2D em AlvoTemplo quando a tag pertence aos alvos reconhecidos; caso contrário retorna null.

Nº	CÓDIGO	O QUE A LINHA FAZ
23	`public static AlvoTemplo Identificar(Collider2D objeto)`	Declara a função Identificar. Transforma um Collider2D em AlvoTemplo quando a tag pertence aos alvos reconhecidos; caso contrário retorna null.
24	`{`	Abre este bloco de instruções.
25	`if (objeto == null)`	Verifica se não foi recebido um colisor válido.
26	`return null;`	Retorna null quando não existe objeto para identificar.
27	`string tag = objeto.gameObject.tag;`	Lê a tag do objeto que contém o colisor recebido.
28	`if (tag.Contains("inimigo") || tag.Contains("destrutivel") || tag == "Limbo")`	Aceita tags que contenham inimigo ou destrutivel e também a tag Limbo.

Nº	CÓDIGO	O QUE A LINHA FAZ
29	`            return objeto.GetComponentInParent<AlvoTemplo>();`	Procura o componente AlvoTemplo no próprio objeto ou em seus pais e o retorna.
30	`        return null;`	Devolve null para tags que não pertencem aos alvos reconhecidos.
31	`    }`	Fecha o bloco deste nível de recuo.

2. Três tipos de projéteis

1, 2 e 3 selecionam o tipo; X dispara. Cada índice escolhe velocidade e recarga. O terceiro recebe impulso vertical e causa explosão em área. O dano é um campo configurado no componente do projétil.

Samurai.cs • DispararMissao

Cria um tiro do tipo selecionado e define direção, velocidade e intervalo. Retorna a cópia ou null.

Nº	CÓDIGO	O QUE A LINHA FAZ
638	`public GameObject DispararMissao()`	Declara a função DispararMissao. Cria um tiro do tipo selecionado e define direção, velocidade e intervalo. Retorna a cópia ou null.
639	`{`	Abre este bloco de instruções.
640	`if (!(combateHabilitado || modoMissao) || missaoEncerrada || ataqueAtivo || defesaAtiva || dashAtivo || carregando != null || !PodeDisparar)`	Impede o disparo sem combate, após o fim da missão, durante katana, defesa, dash, transporte de caixa ou recarga.
641	`return null;`	Retorna null: não foi criado nenhum objeto de tiro.
642	`int tipo = Mathf.Clamp(tipoProjetil, 0, 2);`	Garante que o índice usado nos vetores fique entre 0 e 2.
643	`// Aula 04x01: encontra o modelo pela tag, cria uma copia e limita a cadencia com Invoke.`	Comentário de orientação do código; não é executado pelo Unity.
644	`GameObject modelo = GameObject.FindGameObjectWithTag("projetil" + (tipo + 1));`	Procura o modelo ativo pela tag projetil1, projetil2 ou projetil3. Soma 1 porque os índices começam em zero.
645	`if (modelo == null || modelo.GetComponent<ProjetilTemplo>() == null || modelo.GetComponent<Rigidbody2D>() == null)`	Exige que o modelo exista e tenha os componentes de lógica do projétil e física 2D.
646	`return null;`	Cancela o disparo se faltar qualquer componente necessário.
647	`PodeDisparar = false;`	Bloqueia novos tiros até a recarga terminar.

Nº	CÓDIGO	O QUE A LINHA FAZ
648	`Invoke("AtivarDisparo", new float[]`	Agenda AtivarDisparo; o atraso será escolhido no vetor de números reais a seguir.
649	`{`	Abre a lista de valores do vetor.
650	`.4f, .8f, 1.2f`	Define os intervalos de recarga: 0,4; 0,8; e 1,2 segundo, respectivamente.
651	`}`	Fecha a lista de valores do vetor.
652	`[tipo]);`	Usa o índice do tipo para escolher o intervalo e termina a chamada de Invoke.
653	`float direcao = Renderer.flipX ? - 1 : 1;`	Escolhe -1 para a esquerda ou 1 para a direita, conforme o espelhamento da imagem.
654	`GameObject tiro = Instantiate(modelo, transform.position + new Vector3(direcao * 0.9f, 0, 0), Quaternion.identity);`	Cria uma cópia do modelo 0,9 unidade à frente do jogador, sem rotação adicional.
655	`tiro.GetComponent<ProjetilTemplo>().PrepararDisparo(this);`	Inicializa a cópia como disparo deste jogador e ativa seus componentes.
656	`SomRonin.Tocar(tipo == 2 ? "magia" : "tiro", transform.position);`	Usa som de magia no terceiro tipo e som de tiro nos demais.
657	`tiro.GetComponent<Rigidbody2D>().velocity = new Vector2(direcao * new float[]`	Começa a definir a velocidade do Rigidbody2D. A componente horizontal combina direção e velocidade do vetor.
658	`{`	Abre a lista de valores do vetor.
659	`10, 7, 6`	Define as velocidades horizontais dos três tipos: 10, 7 e 6 unidades por segundo.
660	`}`	Fecha a lista de valores do vetor.
661	`[tipo], tipo == 2 ? 5 : 0);`	Seleciona a velocidade horizontal. O terceiro tipo também recebe velocidade vertical 5; os demais recebem zero.
662	`return tiro;`	Devolve o GameObject recém-criado para quem chamou a função.
663	`}`	Fecha o bloco deste nível de recuo.

Samurai.cs • AtivarDisparo

Libera o disparo quando termina a recarga agendada.

Nº	CÓDIGO	O QUE A LINHA FAZ
665	`void AtivarDisparo()`	Declara a função AtivarDisparo. Libera o disparo quando termina a recarga agendada.
666	`{`	Abre este bloco de instruções.
667	`    PodeDisparar = true;`	Libera o próximo tiro após o atraso agendado pelo Invoke.
668	`}`	Fecha o bloco deste nível de recuo.

ProjetilTemplo.cs • Start

Agenda a duração de vida apenas para cópias reais de projéteis.

Nº	CÓDIGO	O QUE A LINHA FAZ
10	`void Start()`	Declara a função Start. Agenda a duração de vida apenas para cópias reais de projéteis.
11	`{`	Abre este bloco de instruções.
12	`    if (!modelo)`	Só inicia a duração de vida em um tiro real; o objeto modelo deve permanecer disponível.
13	`        Invoke("Expirar", 3.5f);`	Agenda a expiração do tiro para 3,5 segundos depois.
14	`}`	Fecha o bloco deste nível de recuo.

ProjetilTemplo.cs • PrepararDisparo

Recebe o Samurai autor e transforma a cópia do modelo em um tiro ativo.

Nº	CÓDIGO	O QUE A LINHA FAZ
16	`public void PrepararDisparo(Samurai jogador)`	Declara a função PrepararDisparo. Recebe o Samurai autor e transforma a cópia do modelo em um tiro ativo.
17	`{`	Abre este bloco de instruções.
18	`modelo = false;`	Marca esta cópia como projétil real, e não mais como modelo.
19	`dono = jogador;`	Guarda quem disparou, permitindo atribuir o dano ao jogador.
20	`// Apenas o modelo conserva projetil1/2/3; a busca nao seleciona copias em voo.`	Comentário de orientação do código; não é executado pelo Unity.
21	`gameObject.tag = "disparoJogador";`	Troca a tag da cópia para disparoJogador, diferenciando-a dos modelos.
22	`GetComponent<Rigidbody2D>().simulated = true;`	Ativa a simulação física do corpo 2D da cópia.
23	`foreach (Collider2D colisor in GetComponentsInChildren<Collider2D>())`	Percorre os colisores do objeto e dos filhos ativos.
24	`colisor.enabled = true;`	Habilita cada colisor para detectar contatos.
25	`foreach (SpriteRenderer imagem in GetComponentsInChildren<SpriteRenderer>())`	Percorre os renderizadores do objeto e dos filhos ativos.
26	`imagem.enabled = true;`	Habilita cada imagem para mostrar o tiro.
27	`}`	Fecha o bloco deste nível de recuo.

ProjetilTemplo.cs • Expirar

Resolve o fim da duração de vida do projétil.

Nº	CÓDIGO	O QUE A LINHA FAZ
29	`void Expirar()`	Declara a função Expirar. Resolve o fim da duração de vida do projétil.
30	`{`	Abre este bloco de instruções.
31	`    if (tipo == 2 && !hostil)`	Verifica se é o terceiro tipo e se não pertence ao inimigo.
32	`        Explodir();`	Faz esse projétil explodir quando termina sua duração de vida.
33	`    else Destroy(gameObject);`	Nos demais casos, remove o projétil sem explosão.
34	`}`	Fecha o bloco deste nível de recuo.

ProjetilTemplo.cs • OnTriggerEnter2D

Recebe contatos de trigger e reaproveita a regra comum de colisão.

Nº	CÓDIGO	O QUE A LINHA FAZ
36	`void OnTriggerEnter2D(Collider2D objeto)`	Declara a função OnTriggerEnter2D. Recebe contatos de trigger e reaproveita a regra comum de colisão.
37	`{`	Abre este bloco de instruções.
38	`    VerificarColisao(objeto);`	Encaminha o colisor recebido pelo evento de trigger à regra comum de colisão.
39	`}`	Fecha o bloco deste nível de recuo.

ProjetilTemplo.cs • OnCollisionEnter2D

Recebe contatos físicos sólidos e reaproveita a mesma regra de colisão.

Nº	CÓDIGO	O QUE A LINHA FAZ
41	`void OnCollisionEnter2D(Collision2D objeto)`	Declara a função OnCollisionEnter2D. Recebe contatos físicos sólidos e reaproveita a mesma regra de colisão.
42	`{`	Abre este bloco de instruções.
43	`    VerificarColisao(objeto.collider);`	Extrai o colisor do evento de colisão física e o encaminha à mesma regra.
44	`}`	Fecha o bloco deste nível de recuo.

ProjetilTemplo.cs • VerificarColisao

Recebe o colisor atingido e decide entre ignorar, causar dano, explodir e destruir o tiro.

Nº	CÓDIGO	O QUE A LINHA FAZ
46	`void VerificarColisao(Collider2D objeto)`	Declara a função VerificarColisao. Recebe o colisor atingido e decide entre ignorar, causar dano, explodir e destruir o tiro.
47	`{`	Abre este bloco de instruções.
48	`    if (modelo || consumido)`	Ignora objetos modelo e tiros já consumidos por outro contato.
49	`        return;`	Encerra sem processar a colisão.
50	`    string tag = objeto.gameObject.tag;`	Lê a tag do objeto atingido para decidir a reação.
51	`    if (tag == "areaAtaqueEsquerda" || tag == "areaAtaqueDireita")`	Reconhece as duas áreas de ataque da katana.
52	`        return;`	Ignora essas áreas; elas não devem consumir o projétil.
53	`    if (hostil)`	Separa a lógica dos tiros inimigos da lógica dos tiros do jogador.
54	`    {`	Abre este bloco de instruções.

Nº	CÓDIGO	O QUE A LINHA FAZ
55	`if (tag == "Player")`	Testa se o tiro inimigo atingiu o objeto do jogador.
56	`{`	Abre este bloco de instruções.
57	`Samurai jogador = objeto.GetComponent<Samurai>();`	Procura o componente Samurai no objeto atingido.
58	`if (jogador != null)`	Confere se o componente foi encontrado antes de usá-lo.
59	`jogador.ReceberDanoMissao(dano, true);`	Solicita dano e informa true no parâmetro projetil. Isso permite que a defesa bloqueie o golpe.
60	`consumido = true;`	Marca o tiro como consumido, mesmo que o jogador tenha bloqueado o dano.
61	`Destroy(gameObject);`	Solicita a destruição do tiro inimigo.
62	`}`	Fecha o bloco deste nível de recuo.
63	`else if (!objeto.isTrigger && !tag.Contains("inimigo"))`	Caso não seja o jogador, verifica colisão sólida com algo que não tenha inimigo na tag.
64	`{`	Abre este bloco de instruções.
65	`consumido = true;`	Marca o tiro como consumido por esse obstáculo.
66	`Destroy(gameObject);`	Remove o tiro que encontrou o obstáculo sólido.
67	`}`	Fecha o bloco deste nível de recuo.
68	`return;`	Encerra a rotina hostil, evitando executar as regras dos tiros do jogador.
69	`}`	Fecha o bloco deste nível de recuo.
70	`if (tag == "Player" || tag == "disparoJogador" || tag == "disparoInimigo" || tag.StartsWith("projetil"))`	Para tiros do jogador, ignora o próprio jogador, outros disparos e objetos modelo identificados por tag.
71	`return;`	Sai sem consumir o tiro nesses contatos ignorados.
72	`AlvoTemplo alvo = AlvoTemplo.Identificar(objeto);`	Tenta reconhecer o colisor como alvo que pode receber dano.

Nº	CÓDIGO	O QUE A LINHA FAZ
73	`if (alvo != null || !objeto.isTrigger)`	Só reage quando existe alvo válido ou uma superfície sólida.
74	`{`	Abre este bloco de instruções.
75	`if (tipo == 2)`	Verifica se este é o terceiro tipo de projétil.
76	`Explodir();`	Processa a explosão em área desse tipo.
77	`else`	Executa a alternativa quando a condição do if correspondente não é atendida.
78	`{`	Abre este bloco de instruções.
79	`consumido = true;`	Nos outros tipos, marca o tiro como consumido pelo impacto.
80	`if (alvo != null)`	Confere se o impacto atingiu um alvo que aceita dano.
81	`alvo.ReceberDano(dano, dono);`	Envia o dano configurado e o autor do disparo ao alvo.
82	`Destroy(gameObject);`	Remove o tiro comum após atingir alvo ou superfície.
83	`}`	Fecha o bloco deste nível de recuo.
84	`}`	Fecha o bloco deste nível de recuo.
85	`}`	Fecha o bloco deste nível de recuo.

ProjetilTemplo.cs • Explodir

Aplica dano em círculo, uma vez por alvo, e remove o projétil.

Nº	CÓDIGO	O QUE A LINHA FAZ
87	`void Explodir()`	Declara a função Explodir. Aplica dano em círculo, uma vez por alvo, e remove o projétil.
88	`{`	Abre este bloco de instruções.

Nº	CÓDIGO	O QUE A LINHA FAZ
89	`if (modelo || consumido)`	Impede que o modelo exploda ou que um mesmo tiro exploda duas vezes.
90	`return;`	Encerra a tentativa inválida de explosão.
91	`consumido = true;`	Marca o tiro como consumido antes de percorrer os alvos.
92	`SomRonin.Tocar("explosao", transform.position);`	Reproduz o efeito sonoro da explosão.
93	`System.Collections.Generic.HashSet<int> atingidos = new System.Collections.Generic.HashSet<int>();`	Cria um conjunto de identificadores únicos para não repetir dano em um alvo com vários colisores.
94	`foreach (Collider2D colisor in Physics2D.OverlapCircleAll(transform.position, 1.8f))`	Percorre todos os colisores encontrados em um círculo de raio 1,8 unidade ao redor do tiro.
95	`{`	Abre este bloco de instruções.
96	`AlvoTemplo alvo = AlvoTemplo.Identificar(colisor);`	Tenta obter um alvo de combate a partir de cada colisor.
97	`if (alvo != null && atingidos.Add(alvo.GetInstanceID()))`	Aceita apenas alvos válidos ainda não inseridos no conjunto de atingidos.
98	`alvo.ReceberDano(dano, dono);`	Aplica o dano configurado uma vez por alvo, mantendo a autoria do jogador.
99	`}`	Fecha o bloco deste nível de recuo.
100	`Destroy(gameObject);`	Remove o projétil depois de processar a área da explosão.
101	`}`	Fecha o bloco deste nível de recuo.

3. Inimigo dispara por proximidade

O guarda compara a distância com seu alcance e testa obstáculos na linha de visão. Antes de atirar, mostra uma preparação. O chefe combina ataque distante e golpe próximo em uma máquina de estados.

GuardaTemplo.cs • Awake

Obtém componentes e conecta as reações de ferimento e morte aos eventos do alvo.

Nº	CÓDIGO	O QUE A LINHA FAZ
15	`void Awake()`	Declara a função Awake. Obtém componentes e conecta as reações de ferimento e morte aos eventos do alvo.
16	`{`	Abre este bloco de instruções.
17	`imagem = GetComponent<SpriteRenderer>();`	Obtém a imagem usada para virar o guarda à esquerda ou à direita.
18	`animador = GetComponent<Animator>();`	Obtém o Animator que controla suas animações.
19	`alvo = GetComponent<AlvoTemplo>();`	Obtém o componente que controla vida, dano e derrota.
20	`if (alvo != null)`	Confere se o componente de alvo existe antes de conectar eventos.
21	`{`	Abre este bloco de instruções.
22	`alvo.AoReceberDano += Ferir;`	Inscreve Ferir no evento de dano para reagir quando o guarda é atingido.
23	`alvo.AoDerrotar += Morrer;`	Inscreve Morrer no evento de derrota para executar a reação de morte.
24	`}`	Fecha o bloco deste nível de recuo.
25	`}`	Fecha o bloco deste nível de recuo.

GuardaTemplo.cs • Start

Registra a origem da patrulha e encontra o jogador na cena.

Nº	CÓDIGO	O QUE A LINHA FAZ
27	`void Start()`	Declara a função Start. Registra a origem da patrulha e encontra o jogador na cena.
28	`{`	Abre este bloco de instruções.
29	`    inicio = transform.position;`	Guarda a posição inicial como referência da área de patrulha.
30	`    GameObject personagem = GameObject.FindGameObjectWithTag("Player");`	Procura o objeto ativo identificado pela tag Player.
31	`    if (personagem != null)`	Confere se esse objeto foi encontrado.
32	`        jogador = personagem.GetComponent<Samurai>();`	Guarda seu componente Samurai para consultar posição e estado da missão.
33	`}`	Fecha o bloco deste nível de recuo.

GuardaTemplo.cs • Animar

Recebe o nome desejado e atualiza os parâmetros do Animator do guarda.

Nº	CÓDIGO	O QUE A LINHA FAZ
44	`void Animar(string nome)`	Declara a função Animar. Recebe o nome desejado e atualiza os parâmetros do Animator do guarda.
45	`{`	Abre este bloco de instruções.
46	`    if (animador == null || animacao == nome)`	Evita atualizar sem Animator ou repetir a animação já selecionada.
47	`        return;`	Sai porque nenhuma mudança é necessária ou possível.
48	`    animacao = nome;`	Registra o nome da animação solicitada.
49	`    animador.SetBool("andando", nome == "Correndo");`	Ativa o parâmetro andando somente para o estado Correndo.

Nº	CÓDIGO	O QUE A LINHA FAZ
50	`animador.SetBool("estaAtacando", nome == "Ataque");`	Ativa o parâmetro estaAtacando somente para Ataque.
51	`animador.SetBool("ferido", nome == "Ferido");`	Ativa o parâmetro ferido somente para Ferido.
52	`animador.Update(0);`	Força a avaliação do Animator sem avançar seu tempo.
53	`}`	Fecha o bloco deste nível de recuo.

GuardaTemplo.cs • Update

A cada quadro, decide se o guarda segue, prepara um tiro, dispara ou patrulha.

Nº	CÓDIGO	O QUE A LINHA FAZ
69	`void Update()`	Declara a função Update. A cada quadro, decide se o guarda segue, prepara um tiro, dispara ou patrulha.
70	`{`	Abre este bloco de instruções.
71	`if (alvo != null && alvo.Derrotado)`	Verifica se o guarda já foi derrotado.
72	`return;`	Encerra o comportamento de um guarda derrotado.
73	`if (jogador == null || jogador.missaoEncerrada || Time.time < fimFerido)`	Interrompe se não houver jogador, a missão terminou ou o guarda ainda está reagindo ao ferimento.
74	`return;`	Sai antes de patrulhar, perseguir ou disparar.
75	`float distancia = Vector2.Distance(transform.position, jogador.transform.position);`	Calcula a distância em linha reta entre guarda e jogador no plano 2D.
76	`Vector3 anterior = transform.position;`	Salva a posição anterior para detectar movimento ao final.
77	`if (distancia < alcance && VisaoCombate.Livre(transform.position, jogador.transform.position, gameObject, jogador))`	Exige simultaneamente distância menor que o alcance e linha de visão livre.

Nº	CÓDIGO	O QUE A LINHA FAZ
78	`{`	Abre este bloco de instruções.
79	`if (!preparando)`	Só permite mudar a direção antes da preparação do tiro.
80	`imagem.flipX = jogador.transform.position.x < transform.position.x;`	Espelha a imagem quando o jogador está à esquerda do guarda.
81	`if (distancia > 3 && !preparando)`	Decide aproximar-se se a distância passa de 3 unidades e não há tiro sendo preparado.
82	`SeguirPlayer();`	Executa a perseguição limitada à área permitida.
83	`if (!preparando && Time.time >= proximoTiro && projetil != null)`	Exige ausência de preparação, recarga concluída e um modelo de projétil configurado.
84	`{`	Abre este bloco de instruções.
85	`preparando = true;`	Entra na preparação do disparo.
86	`dispararEm = Time.time + .45f;`	Agenda a saída do projétil para 0,45 segundo depois, dando um aviso visual.
87	`animacao = "";`	Limpa a animação registrada para permitir reiniciar Ataque.
88	`Animar("Ataque");`	Seleciona a animação de ataque durante a preparação.
89	`}`	Fecha o bloco deste nível de recuo.
90	`if (preparando && Time.time >= dispararEm)`	Verifica se a preparação terminou e chegou a hora de disparar.
91	`{`	Abre este bloco de instruções.
92	`preparando = false;`	Sai do estado de preparação.
93	`proximoTiro = Time.time + 1.8f;`	Define a próxima oportunidade de tiro para 1,8 segundo depois.
94	`SomRonin.Tocar("tiro", transform.position);`	Reproduz o som do disparo.
95	`float direcao = imagem.flipX ? - 1 : 1;`	Escolhe esquerda (-1) ou direita (1) usando a orientação da imagem.

Nº	CÓDIGO	O QUE A LINHA FAZ
96	`GameObject tiro = Instantiate(projetil, transform.position + Vector3.right * direcao, Quaternion.identity);`	Instancia o projétil uma unidade à frente do guarda, sem rotação adicional.
97	`tiro.GetComponent<Rigidbody2D>().velocity = new Vector2(direcao * 5, 0);`	Aplica velocidade horizontal de 5 unidades por segundo, com velocidade vertical inicial zero.
98	`}`	Fecha o bloco deste nível de recuo.
99	`}`	Fecha o bloco deste nível de recuo.
100	`else`	Executa a alternativa quando a condição do if correspondente não é atendida.
101	`{`	Abre este bloco de instruções.
102	`preparando = false;`	Cancela a preparação quando perde alcance ou visão do jogador.
103	`MovimentoPingPong();`	Retoma o movimento de patrulha de ida e volta.
104	`}`	Fecha o bloco deste nível de recuo.
105	`if (!preparando)`	Só atualiza animação de locomoção quando não está preparando um tiro.
106	`Animar((transform.position - anterior).sqrMagnitude > .000001f ? "Correndo" : "Parado");`	Compara o deslocamento ao quadrado com uma tolerância pequena e escolhe Correndo ou Parado.
107	`}`	Fecha o bloco deste nível de recuo.

GuardaTemplo.cs • SeguirPlayer

Aproxima o guarda do jogador respeitando os limites horizontais da área.

Nº	CÓDIGO	O QUE A LINHA FAZ
109	`void SeguirPlayer()`	Declara a função SeguirPlayer. Aproxima o guarda do jogador respeitando os limites horizontais da área.
110	`{`	Abre este bloco de instruções.
111	`Vector3 destino = new Vector3(Mathf.Clamp(jogador.transform.position.x, inicio.x - 1, inicio.x + distanciaPatrulha), inicio.y, inicio.z);`	Cria um destino com X do jogador limitado entre início menos 1 e início mais a distância de patrulha; mantém Y e Z iniciais.
112	`transform.position = Vector3.MoveTowards(transform.position, destino, velocidade * Time.deltaTime);`	Avança até o destino no máximo velocidade vezes deltaTime, tornando o passo proporcional ao tempo do quadro.
113	`// Conserva a fase da patrulha ao voltar, evitando um teleporte para o inicio.`	Comentário de orientação do código; não é executado pelo Unity.
114	`float deslocamento = Mathf.Clamp(transform.position.x - inicio.x, 0, distanciaPatrulha);`	Mede o deslocamento horizontal em relação ao início e o limita à extensão de patrulha.
115	`TempoDecorrido = imagem.flipX ? 2 * distanciaPatrulha - deslocamento : deslocamento;`	Ajusta a fase do PingPong conforme posição e direção para retomar a patrulha de maneira coerente.
116	`}`	Fecha o bloco deste nível de recuo.

GuardaTemplo.cs • MovimentoPingPong

Produz uma patrulha de ida e volta, proporcional ao tempo de cada quadro.

Nº	CÓDIGO	O QUE A LINHA FAZ
118	`void MovimentoPingPong()`	Declara a função MovimentoPingPong. Produz uma patrulha de ida e volta, proporcional ao tempo de cada quadro.
119	`{`	Abre este bloco de instruções.
120	`TempoDecorrido += Time.deltaTime * velocidade;`	Avança o acumulador de patrulha pela velocidade multiplicada pelo tempo do quadro.
121	`float deslocamento = Mathf.PingPong(TempoDecorrido, distanciaPatrulha);`	Converte o acumulador em deslocamento que vai e volta entre zero e distanciaPatrulha.
122	`Vector3 destino = inicio + Vector3.right * deslocamento;`	Soma esse deslocamento horizontal à posição inicial para obter o destino.
123	`float anterior = transform.position.x;`	Guarda o X atual para descobrir o sentido real do movimento.
124	`// MoveTowards tambem evita saltos se a perseguicao saiu do trecho original.`	Comentário de orientação do código; não é executado pelo Unity.
125	`transform.position = Vector3.MoveTowards(transform.position, destino, velocidade * Time.deltaTime);`	Aproxima a posição do destino com um passo limitado pela velocidade e pelo tempo.
126	`imagem.flipX = transform.position.x < anterior;`	Vira a imagem à esquerda quando o X diminuiu em relação ao quadro anterior.
127	`}`	Fecha o bloco deste nível de recuo.

GuardaTemplo.cs • OnDrawGizmosSelected

Desenha alcance, limites e visão no editor quando o objeto está selecionado e os Gizmos estão visíveis.

Nº	CÓDIGO	O QUE A LINHA FAZ
129	`void OnDrawGizmosSelected()`	Declara a função OnDrawGizmosSelected. Desenha alcance, limites e visão no editor quando o objeto está selecionado e os Gizmos estão visíveis.
130	`{`	Abre este bloco de instruções.
131	`    Gizmos.color = Color.yellow;`	Seleciona amarelo para o próximo desenho de depuração.
132	`    Gizmos.DrawWireSphere(transform.position, alcance);`	Desenha o alcance de detecção como uma esfera de arame; na cena 2D, aparece como referência circular.
133	`    Vector3 origem = Application.isPlaying ? inicio : transform.position;`	Durante o jogo usa o início salvo; fora dele usa a posição atual para desenhar a patrulha.
134	`    Gizmos.color = Color.cyan;`	Troca a cor do desenho para ciano.
135	`    Gizmos.DrawLine(origem - Vector3.right, origem + Vector3.right * distanciaPatrulha);`	Desenha o segmento horizontal que representa os limites usados pela perseguição.
136	`    if (jogador != null)`	Só desenha a ligação com o jogador quando sua referência existe.
137	`    {`	Abre este bloco de instruções.
138	`        bool visivel = Vector2.Distance(transform.position, jogador.transform.position) < alcance && VisaoCombate.Livre(transform.position, jogador.transform.position, gameObject, jogador);`	Recalcula alcance e linha de visão para mostrar se o guarda consegue ver o jogador.
139	`        Gizmos.color = visivel ? Color.green : Color.gray;`	Usa verde quando o jogador está visível e cinza quando não está.
140	`        Gizmos.DrawLine(transform.position, jogador.transform.position);`	Desenha uma linha de diagnóstico entre guarda e jogador. Esse desenho não move nenhum personagem.
141	`    }`	Fecha o bloco deste nível de recuo.
142	`}`	Fecha o bloco deste nível de recuo.

VisaoCombate.cs • Livre

Recebe origem, destino, emissor e jogador. Retorna bool indicando se há visão sem obstáculos sólidos.

Nº	CÓDIGO	O QUE A LINHA FAZ
5	`public static bool Livre(Vector2 origem, Vector2 destino, GameObject emissor, Samurai jogador)`	Declara a função Livre. Recebe origem, destino, emissor e jogador. Retorna bool indicando se há visão sem obstáculos sólidos.
6	`{`	Abre este bloco de instruções.
7	`foreach (RaycastHit2D hit in Physics2D.LinecastAll(origem, destino))`	Percorre os resultados de uma consulta física ao longo do segmento entre origem e destino.
8	`{`	Abre este bloco de instruções.
9	`Collider2D c = hit.collider;`	Obtém o colisor associado ao resultado atual.
10	`if (c == null || c.isTrigger || c.transform.IsChildOf(emissor.transform))`	Ignora resultados sem colisor, triggers e colisores pertencentes à hierarquia de quem está olhando.
11	`continue;`	Pula para o próximo resultado da consulta.
12	`if (jogador != null && c.GetComponentInParent<Samurai>() == jogador)`	Verifica se o colisor pertence ao próprio jogador que se deseja enxergar.
13	`continue;`	Ignora o jogador como obstáculo e continua procurando bloqueios.
14	`return false;`	Qualquer outro colisor sólido bloqueia a visão; retorna falso imediatamente.
15	`}`	Fecha o bloco deste nível de recuo.
16	`return true;`	Sem encontrar bloqueio, retorna verdadeiro para indicar visão livre.
17	`}`	Fecha o bloco deste nível de recuo.

OldSamurai.cs • Enxerga

Reúne os critérios que permitem ao chefe detectar e atacar o jogador.

Nº	CÓDIGO	O QUE A LINHA FAZ
72	`bool Enxerga()`	Declara a função Enxerga. Reúne os critérios que permitem ao chefe detectar e atacar o jogador.
73	`{`	Abre este bloco de instruções.
74	`return jogador != null && jogador.modoMissao && !jogador.missaoEncerrada && Mathf.Abs(jogador.transform.position.x - inicio.x) <= distanciaPatrulha + deteccao && Vector2.Distance(Centro, jogador.transform.position) <= deteccao && VisaoCombate.Livre(Centro, jogador.transform.position, gameObject, jogador);`	Exige jogador existente, modo missão ativo, missão não encerrada, limite horizontal respeitado, distância dentro de deteccao e linha de visão sem bloqueios.
75	`}`	Fecha o bloco deste nível de recuo.

OldSamurai.cs • Trocar

Recebe novo estado, duração e animação; sincroniza a inteligência do chefe com sua apresentação visual.

Nº	CÓDIGO	O QUE A LINHA FAZ
77	`void Trocar(Estado novo, float duracao, string animacao)`	Declara a função Trocar. Recebe novo estado, duração e animação; sincroniza a inteligência do chefe com sua apresentação visual.
78	`{`	Abre este bloco de instruções.
79	`if (modoMago && animacao == "Ataque" && !ataqueDistante)`	Reconhece o ataque próximo do mago: modoMago ativo, animação de ataque solicitada e ataqueDistante falso.
80	`animacao = "Ataque2";`	Substitui a animação por Ataque2 para diferenciar o golpe próximo.
81	`estado = novo;`	Armazena o novo estado da inteligência do chefe.

Nº	CÓDIGO	O QUE A LINHA FAZ
82	`fimEstado = Time.time + duracao;`	Calcula quando esse estado deve terminar, somando a duração ao relógio do jogo.
83	`if (animador != null)`	Só altera animações se o Animator estiver disponível.
84	`{`	Abre este bloco de instruções.
85	`animador.speed = 1;`	Restaura a velocidade normal antes de atualizar os parâmetros.
86	`animador.SetBool("andando", animacao == "Correndo");`	Ativa andando quando a animação solicitada é Correndo.
87	`animador.SetBool("estaAtacando", animacao == "Ataque");`	Ativa estaAtacando quando a animação solicitada é Ataque.
88	`animador.SetBool("ferido", animacao == "Ferido");`	Ativa ferido quando a animação solicitada é Ferido.
89	`animador.Update(0);`	Avalia o Animator imediatamente, sem avanço de tempo.
90	`// A FSM conserva o aviso e o quadro exato de ataque solicitado para o chefe.`	Comentário de orientação do código; não é executado pelo Unity.
91	`animador.speed = novo == Estado.Preparando ? 0 : 1;`	Pausa a animação na preparação; nos demais estados mantém velocidade normal.
92	`animador.Play(animacao, 0, novo == Estado.Preparando ? 1f / 7f : 0);`	Seleciona diretamente a animação na camada 0. Durante a preparação posiciona em 1/7 do ciclo; nos demais casos começa em zero.
93	`animador.Update(0);`	Aplica imediatamente o estado selecionado pelo Play.
94	`}`	Fecha o bloco deste nível de recuo.
95	`}`	Fecha o bloco deste nível de recuo.

OldSamurai.cs • FixedUpdate

Executa a máquina de estados do chefe nos passos de física: patrulha, perseguição, aviso, ataque e recuperação.

Nº	CÓDIGO	O QUE A LINHA FAZ
97	`void FixedUpdate()`	Declara a função FixedUpdate. Executa a máquina de estados do chefe nos passos de física: patrulha, perseguição, aviso, ataque e recuperação.
98	`{`	Abre este bloco de instruções.
99	`if (estado == Estado.Derrotado || jogador == null || jogador.missaoEncerrada)`	Impede ações quando o chefe morreu, não há jogador ou a missão terminou.
100	`return;`	Encerra esta atualização física sem realizar ações.
101	`if (estado == Estado.Ferido || estado == Estado.Recuperando)`	Trata os estados de ferimento e recuperação, que têm duração limitada.
102	`{`	Abre este bloco de instruções.
103	`if (Time.time >= fimEstado)`	Verifica se o tempo do estado terminou.
104	`Trocar(Estado.Patrulha, 0, "Parado");`	Volta ao estado de patrulha com animação parada inicialmente.
105	`return;`	Sai para não decidir um novo ataque no mesmo passo dessa recuperação.
106	`}`	Fecha o bloco deste nível de recuo.
107	`if (estado == Estado.Preparando)`	Trata o período de aviso que antecede o ataque.
108	`{`	Abre este bloco de instruções.
109	`if (Time.time >= fimEstado)`	Confere se o aviso terminou.
110	`{`	Abre este bloco de instruções.
111	`golpeAplicado = false;`	Libera a aplicação de um golpe neste novo ataque.
112	`Trocar(Estado.Atacando, (modoMago ? 8f : 7f) / 12f, "Ataque");`	Entra no ataque por 8/12 de segundo no mago ou 7/12 no outro modo, selecionando sua animação.

Nº	CÓDIGO	O QUE A LINHA FAZ
113	`        }`	Fecha o bloco deste nível de recuo.
114	`        return;`	Encerra o tratamento da preparação neste passo.
115	`    }`	Fecha o bloco deste nível de recuo.
116	`    if (estado == Estado.Atacando)`	Trata o estado em que o ataque já está em execução.
117	`    {`	Abre este bloco de instruções.
118	`        // O golpe acontece no quarto quadro, depois do aviso e da preparacao da lamina.`	Comentário de orientação do código; não é executado pelo Unity.
119	`        if (!golpeAplicado && Time.time >= fimEstado - 4f / 12f)`	Dispara o efeito do golpe uma única vez, a partir do instante correspondente a 4/12 de segundo antes do fim do ataque.
120	`        {`	Abre este bloco de instruções.
121	`            golpeAplicado = true;`	Registra que o golpe já foi aplicado, evitando repetições nos próximos passos.
122	`            AplicarGolpe();`	Executa o ataque distante ou próximo conforme a escolha feita anteriormente.
123	`        }`	Fecha o bloco deste nível de recuo.
124	`        if (Time.time >= fimEstado)`	Testa se a animação temporizada de ataque terminou.
125	`            Trocar(Estado.Recuperando, recuperacao, "Parado");`	Entra na recuperação pelo tempo configurado, com animação parada.
126	`        return;`	Sai para manter a lógica de ataque separada da decisão de patrulha.
127	`    }`	Fecha o bloco deste nível de recuo.
128	`    if (Enxerga())`	Só decide perseguição ou ataque se as condições de visão forem atendidas.
129	`    {`	Abre este bloco de instruções.
130	`        float dx = jogador.transform.position.x - corpo.position.x;`	Calcula a diferença horizontal entre jogador e corpo do chefe.

Nº	CÓDIGO	O QUE A LINHA FAZ
131	`imagem.flipX = dx < 0;`	Espelha a imagem quando essa diferença indica jogador à esquerda.
132	`float distancia = Vector2.Distance(Centro, jogador.transform.position);`	Mede a distância do centro de combate do chefe ao jogador.
133	`if (distancia <= alcanceKatana || distancia >= 3.2f)`	Permite atacar muito perto, dentro do alcance de golpe, ou longe, a partir de 3,2 unidades.
134	`{`	Abre este bloco de instruções.
135	`ataqueDistante = distancia > alcanceKatana;`	Escolhe ataque distante quando a distância supera o alcance do golpe próximo.
136	`ladoAtaque = imagem.flipX ? - 1 : 1;`	Guarda o lado do ataque para preservar a direção durante sua preparação.
137	`Trocar(Estado.Preparando, aviso, "Ataque");`	Inicia o aviso usando a duração configurada no campo aviso.
138	`}`	Fecha o bloco deste nível de recuo.
139	`else`	Executa a alternativa quando a condição do if correspondente não é atendida.
140	`{`	Abre este bloco de instruções.
141	`if (estado != Estado.Perseguindo)`	Evita repetir a troca se o chefe já está perseguindo.
142	`Trocar(Estado.Perseguindo, 0, "Correndo");`	Entra em perseguição com animação de corrida.
143	`Mover(Mathf.Clamp(jogador.transform.position.x, inicio.x - distanciaPatrulha, inicio.x + distanciaPatrulha));`	Solicita movimento até o X do jogador, limitado aos extremos da área de patrulha.
144	`}`	Fecha o bloco deste nível de recuo.
145	`}`	Fecha o bloco deste nível de recuo.
146	`else`	Executa a alternativa quando a condição do if correspondente não é atendida.
147	`{`	Abre este bloco de instruções.
148	`if (estado != Estado.Patrulha)`	Quando não vê o jogador, verifica se precisa voltar ao estado de patrulha.

Nº	CÓDIGO	O QUE A LINHA FAZ
149	`Trocar(Estado.Patrulha, 0, "Correndo");`	Ativa patrulha com animação de corrida.
150	`float destino = inicio.x + ladoPatrulha * distanciaPatrulha;`	Calcula o extremo da patrulha correspondente à direção atual.
151	`if (Mathf.Abs(corpo.position.x - destino) < .08f)`	Testa se chegou a menos de 0,08 unidade desse extremo.
152	`ladoPatrulha *= - 1;`	Multiplica o sinal por -1, invertendo o sentido da patrulha.
153	`Mover(inicio.x + ladoPatrulha * distanciaPatrulha);`	Solicita movimento em direção ao extremo correspondente ao novo sentido.
154	`}`	Fecha o bloco deste nível de recuo.
155	`}`	Fecha o bloco deste nível de recuo.

OldSamurai.cs • AplicarGolpe

Materializa o disparo distante ou a área retangular do ataque próximo do mago.

Nº	CÓDIGO	O QUE A LINHA FAZ
178	`void AplicarGolpe()`	Declara a função AplicarGolpe. Materializa o disparo distante ou a área retangular do ataque próximo do mago.
179	`{`	Abre este bloco de instruções.
180	`if (!Enxerga())`	Revalida a visão no instante do golpe, pois o jogador pode ter saído do alcance ou se escondido.
181	`return;`	Cancela o golpe se o chefe não consegue mais enxergar o jogador.
182	`AtaqueExecutado = ataqueDistante ? 1 : 2;`	Registra 1 para ataque distante ou 2 para golpe próximo, permitindo consultar qual foi executado.
183	`SomRonin.Tocar(ataqueDistante ? "magia" : "katana", transform.position);`	Toca magia no ataque distante e katana no ataque próximo.

Nº	CÓDIGO	O QUE A LINHA FAZ
184	`if (ataqueDistante)`	Seleciona a execução do disparo distante.
185	`{`	Abre este bloco de instruções.
186	`if (projetil == null)`	Verifica se falta o modelo de projétil do chefe.
187	`return;`	Cancela o disparo se o modelo não foi configurado.
188	`Vector2 posicao = Centro + Vector2.right * ladoAtaque * .7f;`	Calcula uma posição de nascimento 0,7 unidade à frente do centro do chefe.
189	`if (!VisaoCombate.Livre(Centro, posicao, gameObject, jogador))`	Testa se existe obstáculo entre o centro e o ponto de nascimento do tiro.
190	`return;`	Cancela para evitar criar o tiro do outro lado de uma parede próxima.
191	`GameObject tiro = Instantiate(projetil, posicao, Quaternion.identity);`	Cria uma cópia do projétil na posição calculada, sem rotação adicional.
192	`Rigidbody2D rb = tiro.GetComponent<Rigidbody2D>();`	Obtém o corpo físico 2D dessa cópia.
193	`if (rb != null)`	Confere se o corpo existe antes de mudar a velocidade.
194	`rb.velocity = Vector2.right * ladoAtaque * 4.5f;`	Lança o projétil horizontalmente a 4,5 unidades por segundo no lado guardado.
195	`}`	Fecha o bloco deste nível de recuo.
196	`else`	Executa a alternativa quando a condição do if correspondente não é atendida.
197	`{`	Abre este bloco de instruções.
198	`Collider2D corpoJogador = jogador.GetComponent<Collider2D>();`	Obtém o colisor principal do jogador para reconhecer o corpo atingível.
199	`Vector2 centroGolpe = Centro + Vector2.right * ladoAtaque * .85f;`	Coloca o centro do golpe próximo 0,85 unidade à frente do chefe.
200	`foreach (Collider2D c in Physics2D.OverlapBoxAll(centroGolpe, new Vector2(1.2f, 1.3f), 0))`	Consulta colisores em um retângulo de 1,2 por 1,3 unidade, sem rotação, e percorre os resultados.

Nº	CÓDIGO	O QUE A LINHA FAZ
201	`if (c.CompareTag("Player") && c == corpoJogador)`	Aceita somente o colisor principal do jogador identificado também pela tag Player.
202	`{`	Abre este bloco de instruções.
203	`jogador.ReceberDanoMissao(1, false);`	Solicita 1 ponto de dano não classificado como projétil. A defesa contra disparos não bloqueia este golpe.
204	`break;`	Encerra a busca após atingir o jogador uma vez.
205	`}`	Fecha o bloco deste nível de recuo.
206	`}`	Fecha o bloco deste nível de recuo.
207	`}`	Fecha o bloco deste nível de recuo.

4. Defesa por tecla e animação

Segurar F solicita defesa. O estado ativa o parâmetro defendendo do Animator. O dano identificado como projétil é bloqueado; o golpe próximo do chefe não é bloqueado por essa regra.

Samurai.cs • DefinirDefesa

Recebe bool ativa e aceita a defesa somente quando o estado do jogador permite.

Nº	CÓDIGO	O QUE A LINHA FAZ
670	`public void DefinirDefesa(bool ativa)`	Declara a função DefinirDefesa. Recebe bool ativa e aceita a defesa somente quando o estado do jogador permite.
671	`{`	Abre este bloco de instruções.
672	`defesaAtiva = ativa && !ataqueAtivo && !dashAtivo && carregando == null && !missaoEncerrada;`	Ativa a defesa apenas quando solicitada e sem ataque, dash, caixa carregada ou missão encerrada.
673	`}`	Fecha o bloco deste nível de recuo.

Samurai.cs • ReceberDanoMissao

Recebe a quantidade de dano e se ele veio de projétil; retorna bool indicando se foi aceito.

Nº	CÓDIGO	O QUE A LINHA FAZ
675	`public bool ReceberDanoMissao(int dano, bool projetil)`	Declara a função ReceberDanoMissao. Recebe a quantidade de dano e se ele veio de projétil; retorna bool indicando se foi aceito.
676	`{`	Abre este bloco de instruções.
677	`if (!(combateHabilitado || modoMissao) || dano <= 0 || missaoEncerrada || Time.time < invulneravelAte || (projetil && defesaAtiva))`	Rejeita dano fora de combate, não positivo, após o fim, durante invulnerabilidade ou vindo de projétil enquanto a defesa está ativa.

Nº	CÓDIGO	O QUE A LINHA FAZ
678	`return false;`	Informa que o dano foi rejeitado e não reduz vidas.
679	`if (!feedbackDano && Renderer != null)`	Antes de iniciar um novo efeito, verifica se pode guardar a cor atual do personagem.
680	`corAntesDano = Renderer.color;`	Salva a cor que será restaurada depois do piscar.
681	`feedbackDano = true;`	Ativa o controle do efeito visual de dano.
682	`SomRonin.Tocar("dano", transform.position);`	Reproduz o som de dano no personagem.
683	`ContadorVidas = Mathf.Max(0, ContadorVidas - dano);`	Subtrai o dano das vidas e impede que o resultado fique negativo.
684	`invulneravelAte = Time.time + 1.2f;`	Concede 1,2 segundo de invulnerabilidade após o golpe aceito.
685	`if (UITextVidas != null)`	Confere se há referência ao texto de vidas.
686	`UITextVidas.text = ContadorVidas.ToString();`	Converte o número de vidas em texto e atualiza a interface.
687	`if (ContadorVidas == 0 && modoMissao)`	Testa se as vidas acabaram durante uma missão.
688	`EncerrarMissao(false);`	Encerra a missão com resultado de derrota; false significa que não houve vitória.
689	`return true;`	Informa que o dano foi aceito.
690	`}`	Fecha o bloco deste nível de recuo.

AnimacaoSamurai.cs • LateUpdate

Escolhe a animação do jogador depois das atualizações do quadro, incluindo a animação de defesa.

Nº	CÓDIGO	O QUE A LINHA FAZ
51	`void LateUpdate()`	Declara a função LateUpdate. Escolhe a animação do jogador depois das atualizações do quadro, incluindo a animação de defesa.
52	`{`	Abre este bloco de instruções.
53	`NoChao = VerificarChao();`	Atualiza se o personagem está apoiado no chão, usando a rotina de detecção de solo.
54	`bool novoPulo = ultimoPulo != jogador.SequenciaPulo;`	Compara o número de sequência dos pulos para reconhecer um novo salto, inclusive no ar.
55	`ultimoPulo = jogador.SequenciaPulo;`	Guarda a sequência atual para a comparação no próximo quadro.
56	`string proximo = jogador.ataqueAtivo ? "Atacando" : jogador.defesaAtiva ? "Defendendo" : jogador.dashAtivo ? "Dash" : !NoChao ? "Pulando" : Mathf.Abs(corpo.velocity.x) > .1f ? (jogador.correndo ? "Correndo" : "Andando") : "Parado";`	Escolhe por prioridade: ataque, defesa, dash, pulo, corrida ou caminhada se houver movimento horizontal, e finalmente parado.
57	`bool mudou = estado != proximo;`	Compara a animação escolhida com o estado anterior para detectar mudança.
58	`// Aulas 03x05 e 04x04: os bools comandam transicoes sem Has Exit Time.`	Comentário de orientação do código; não é executado pelo Unity.
59	`animador.speed = 1;`	Restaura a velocidade normal do Animator antes de avaliar os parâmetros.
60	`animador.SetBool("andando", proximo == "Andando");`	Ativa andando somente quando a escolha foi Andando.
61	`animador.SetBool("correndo", proximo == "Correndo");`	Ativa correndo somente quando a escolha foi Correndo.
62	`animador.SetBool("pulando", proximo == "Pulando");`	Ativa pulando somente quando a escolha foi Pulando.
63	`animador.SetBool("estaAtacando", proximo == "Atacando");`	Ativa estaAtacando somente quando a escolha foi Atacando.

Nº	CÓDIGO	O QUE A LINHA FAZ
64	`animador.SetBool("defendendo", proximo == "Defendendo");`	Ativa defendendo quando a defesa foi escolhida. Este parâmetro liga a lógica da tecla à animação.
65	`animador.SetBool("dash", proximo == "Dash");`	Ativa dash somente quando a escolha foi Dash.
66	`if (mudou)`	Executa a atualização imediata apenas quando mudou de estado.
67	`{`	Abre este bloco de instruções.
68	`estado = proximo;`	Guarda o nome do novo estado de animação.
69	`animador.Update(0);`	Avalia o Animator para refletir a mudança sem avançar o tempo.
70	`}`	Fecha o bloco deste nível de recuo.
71	`// Extensoes do jogo: reiniciar apenas um NOVO pulo, ou usar a pose de queda`	Comentário de orientação do código; não é executado pelo Unity.
72	`// ao sair de uma beirada. A descida de um pulo normal nunca o reinicia.`	Comentário de orientação do código; não é executado pelo Unity.
73	`if (proximo == "Pulando" && ((novoPulo && !mudou) || (mudou && corpo.velocity.y <= 0)))`	Reposiciona a animação de pulo em um novo salto dentro do mesmo estado ou quando entra nesse estado já caindo.
74	`{`	Abre este bloco de instruções.
75	`animador.Play("Pulando", 0, novoPulo ? 0 : .9f);`	Recomeça Pulando do início em um novo salto; se já está caindo, posiciona em 90% do ciclo, na camada 0.
76	`animador.Update(0);`	Aplica imediatamente a posição de animação escolhida.
77	`}`	Fecha o bloco deste nível de recuo.

Nº	CÓDIGO	O QUE A LINHA FAZ
78	`PuloPausado = proximo == "Pulando" && corpo.velocity.y <= 0;`	Marca o pulo como pausado quando a animação é Pulando e a velocidade vertical não é positiva.
79	`animador.speed = PuloPausado ? 0 : (proximo == "Dash" ? 1.6f : 1);`	Pausa o ciclo durante a descida; caso contrário, acelera o dash para 1,6 vez ou usa velocidade normal.
80	`}`	Fecha o bloco deste nível de recuo.

5. Objetos destrutíveis

AlvoTemplo centraliza vida e dano. Ao chegar a zero, desliga colisores, libera um prêmio quando configurado, avisa os ouvintes do evento de derrota e destrói o objeto.

AlvoTemplo.cs • Awake

Guarda a imagem e sua cor inicial para produzir os efeitos de dano.

Nº	CÓDIGO	O QUE A LINHA FAZ
33	`void Awake()`	Declara a função Awake. Guarda a imagem e sua cor inicial para produzir os efeitos de dano.
34	`{`	Abre este bloco de instruções.
35	`imagem = GetComponent<SpriteRenderer>();`	Obtém o componente responsável por desenhar a imagem deste alvo.
36	`if (imagem != null)`	Verifica se o componente foi encontrado.
37	`corOriginal = imagem.color;`	Guarda sua cor original para restaurar após o feedback de dano.
38	`}`	Fecha o bloco deste nível de recuo.

AlvoTemplo.cs • Update

Atualiza o destaque de dano e a transparência durante o desaparecimento do alvo.

Nº	CÓDIGO	O QUE A LINHA FAZ
40	`void Update()`	Declara a função Update. Atualiza o destaque de dano e a transparência durante o desaparecimento do alvo.
41	`{`	Abre este bloco de instruções.
42	`if (imagem == null)`	Testa se não existe imagem para atualizar.

Nº	CÓDIGO	O QUE A LINHA FAZ
43	`            return;`	Encerra a atualização visual quando não há SpriteRenderer.
44	`        Color cor = Time.time < fimFeedback ? new Color(1, .35f, .35f, 1) : corOriginal;`	Escolhe vermelho claro durante o feedback de dano ou a cor original quando ele termina.
45	`        if (destruido && atrasoDestruicao > 0)`	Verifica se o alvo morreu e existe tempo reservado ao desaparecimento gradual.
46	`            cor.a = corOriginal.a * (1 - Mathf.Clamp01((Time.time - morteEm) / atrasoDestruicao));`	Reduz a transparência conforme a fração de tempo transcorrido. Clamp01 limita essa fração entre 0 e 1.
47	`        imagem.color = cor;`	Aplica a cor calculada à imagem do alvo.
48	`    }`	Fecha o bloco deste nível de recuo.

AlvoTemplo.cs • ReceberDano

Recebe a quantidade e o Samurai autor; processa dano, morte, prêmio e eventos do alvo.

Nº	CÓDIGO	O QUE A LINHA FAZ
50	`    public void ReceberDano(int quantidade, Samurai autor)`	Declara a função ReceberDano. Recebe a quantidade e o Samurai autor; processa dano, morte, prêmio e eventos do alvo.
51	`    {`	Abre este bloco de instruções.
52	`        if (destruido || quantidade <= 0 || (autor != null && autor.missaoEncerrada))`	Ignora dano se o alvo já foi destruído, a quantidade não é positiva ou o autor já encerrou a missão.
53	`            return;`	Encerra sem modificar a vida.
54	`        SomRonin.Tocar("impacto", transform.position);`	Toca o efeito de impacto na posição atingida.
55	`        vida = Mathf.Max(0, vida - quantidade);`	Subtrai a quantidade de dano e limita a vida mínima a zero.
56	`        fimFeedback = Time.time + .12f;`	Mantém o destaque de dano por 0,12 segundo.
57	`        if (vida > 0)`	Verifica se o alvo sobreviveu ao golpe.

Nº	CÓDIGO	O QUE A LINHA FAZ
58	`{`	Abre este bloco de instruções.
59	`if (AoReceberDano != null)`	Confere se existe alguma função inscrita no evento de dano.
60	`AoReceberDano();`	Notifica os ouvintes, permitindo que o inimigo reaja ao ferimento.
61	`return;`	Termina aqui para não executar a lógica de morte de um alvo vivo.
62	`}`	Fecha o bloco deste nível de recuo.
63	`destruido = true;`	Marca a destruição e impede que novos golpes processem a morte novamente.
64	`morteEm = Time.time;`	Guarda o instante da morte para calcular o desaparecimento visual.
65	`foreach (Collider2D c in GetComponentsInChildren<Collider2D>())`	Percorre os colisores deste objeto e de seus filhos ativos.
66	`c.enabled = false;`	Desativa cada colisor para que o alvo morto não continue bloqueando ou recebendo contatos.
67	`if (premio != null)`	Confere se há um prêmio configurado.
68	`{`	Abre este bloco de instruções.
69	`premio.transform.SetParent(null, true);`	Remove o prêmio da hierarquia do alvo, preservando sua posição no mundo. Assim ele não é destruído junto com o pai.
70	`premio.SetActive(true);`	Ativa o prêmio para aparecer e poder ser coletado.
71	`}`	Fecha o bloco deste nível de recuo.
72	`if (CompareTag("inimigo01") && autor != null && autor.modoMissao)`	Conta a derrota somente para a tag inimigo01, com autor conhecido e missão ativa nesse modo.
73	`autor.guardasDerrotados++;`	Acrescenta um ao contador de guardas derrotados do autor.
74	`if (AoDerrotar != null)`	Verifica se há funções inscritas no evento de derrota.

Nº	CÓDIGO	O QUE A LINHA FAZ
75	`AoDerrotar();`	Avisa os ouvintes da morte; a reação específica do guarda ou chefe é tratada por eles.
76	`Destroy(gameObject, Mathf.Max(0, atrasoDestruicao));`	Agenda a destruição do objeto após o atraso configurado, nunca usando um atraso negativo.
77	`}`	Fecha o bloco deste nível de recuo.

Roteiro para explicar o projeto

1. Mostre o início do fluxo: a tecla Z chega a AtacarKatana; X chega a DispararMissao; F chega a DefinirDefesa.

2. Mostre a validação: o jogador não pode atacar em qualquer estado. Os testes de recarga, missão e ações simultâneas impedem conflitos.

3. Mostre o contato: a espada usa triggers; os projéteis encaminham colisões à mesma regra; ambos encontram um AlvoTemplo.

4. Mostre a consequência: ReceberDano reduz a vida e produz feedback. Ao zerar, desativa colisores, libera prêmio e avisa a derrota.

5. Explique a IA: distância determina proximidade, LinecastAll verifica obstáculos, temporizadores separam aviso e disparo. Os Gizmos apenas ajudam a enxergar esses limites no editor.

6. Diferencie lógica e animação: defesaAtiva decide o bloqueio; o parâmetro defendendo apresenta esse estado visualmente. Uma animação sozinha não bloqueia dano.

Funções de apoio citadas

SomRonin.Tocar reproduz efeitos de áudio. VerificarBonusAndar e IniciarDash pertencem às mecânicas auxiliares do Samurai. VerificarChao identifica apoio para a animação. EncerrarMissao recebe o resultado da partida. Ferir e Morrer são reações conectadas aos eventos do alvo. Mover realiza o deslocamento do chefe. Esses apoios aparecem nas chamadas, mas seus corpos não fazem parte da seleção de funções deste guia.

Origem e verificação

Requisitos: ficha “4BIM_JOGOS_PROJETO”, disponibilizada no material do professor Hélio Esperidião. A ficha também pede um jogo completo e aplicação dos conteúdos anteriores; esta seleção trata especificamente dos cinco itens de combate.

Código: Assets/Scripts do projeto SamuraiBoy_Bimestres34, com o título de jogo Ronin`s Path. Os números de linha correspondem à versão formatada em 04/11/2026. Foram organizados 22 scripts de execução com recuos de quatro espaços e separação de instruções. A comparação dos tokens antes/depois confirmou preservação dos comandos e operadores; a compilação no Unity terminou com código de saída zero.

Este é um guia da implementação do projeto, e não uma transcrição dos slides ou uma certificação da avaliação do professor.